

Grundlagen der Informatik (SimTech)

Übungsblatt 1

Abgabe bis *Freitag* 24.10., 14:00

Abgabe in Dreierteams – für dieses Blatt bitte *eine* PDF-Datei im Abgabeobjekt in der Ilias-Gruppe der Tutorübung hochladen, mit den Namen der Bearbeiter*innen im Dateinamen

Aufgaben zur Vorlesung am 22.10.

1.1 Ausdrücke und Zuweisungen

Wir betrachten hier folgende Varianten, um Berechnungen aufzuschreiben:

- Form 1 (Allgemeine Zuweisung ohne Funktionsaufrufe): Eine Anweisung der Form $\alpha = \beta$, wobei α ein Variablenname ist und β ein Ausdruck aus
 - Literalen,
 - Variablennamen,
 - Operatoren und
 - Klammern (letztere nur, wo nötig),

aber keinen Funktionsaufrufen.

Beispiel: `a = (b+2)*(c+3)`

- Form 2 (Dreiadressform): Eine oder mehrere Anweisungen der Form $\alpha = \beta \text{ op } \gamma$ oder $\alpha = \text{op } \beta$, wobei
 - α ein Variablenname ist,
 - β und γ Variablennamen oder Literale sind und
 - op ein Operator ist.

Beispiel: `a = b+2`
`c = -a`

- Form 3 (Zuweisung mit Funktionsaufrufen): Eine Anweisung der Form $\alpha = \beta$, wobei α ein Variablenname ist und β ein Ausdruck aus
 - Literalen,
 - Variablennamen und
 - (geschachtelten) Aufrufen der Funktionen
 - * `add`, `sub`, `mul`, `floordiv` mit jeweils zwei Parametern, die den zweistelligen Operatoren `+`, `-`, `*`, `//` entsprechen sowie

- * `neg` mit einem Parameter, die dem einstelligen Operator `-` der Negation entspricht.

Beispiel: `a = add(mul(b, 2), neg(c))`

Diese Funktionen setzen wir als gegeben voraus – zum Ausprobieren: Im Paket `operator` gibt es die, Sie können sie mittels

```
from operator import add, sub, mul, floordiv, neg
```

importieren.

- Form 4 (Zuweisung mit Methodenaufrufen): Eine Anweisung der Form $\alpha = \beta$, wobei α ein Variablenname ist und β ein Ausdruck aus

- Literalen,
- Variablennamen und
- (geschachtelten) Aufrufen folgender Methoden auf `int`-Objekten:
 - * `__add__`, `__sub__`, `__mul__`, `__floordiv__` mit jeweils einem Parameter, die den zweistelligen Operatoren `+`, `-`, `*`, `//` entsprechen sowie
 - * `__neg__` ohne Parameter, die dem einstelligen Operator `-` der Negation entspricht.

Beispiel:

```
a = b.__mul__(c.__add__(3).__mul__(4).__add__(d.__neg__()))
```

Technischer Hinweis: Wenn Sie eine der Methoden auf einem `int`-Literal aufrufen wollen, brauchen Sie Klammern, damit der Punkt nicht als Dezimalpunkt gelesen wird: `(7).__add__(3)`

Wir gehen in den folgenden Teilaufgaben davon aus, dass die „Eingangsgrößen“, d.h., die vorkommenden Variablen, denen im Codeausschnitt kein Wert zugewiesen wird, je einen Wert vom Typ `int` enthalten.

Formen Sie die gegebenen Codestücke in die jeweils drei anderen Formen um.

- `t1 = b+c`
`a = t1*2`
- `a = (b+2)*(c+3)`
- `a = sub(b, sub(sub(c, d), e))`
- `a = c.__mul__(d.__neg__().__add__(2))`

1.2 Mehr Ausdrücke

Gegenüber Aufgabe 1.1 kommt nun der Datentyp `bool` der Wahrheitswerte hinzu, mit den Literalen `True` und `False` sowie jede Menge neuer Operatoren:

- Vergleichsoperatoren, mit denen wir z.B. zwei `int`-Werte vergleichen können und ein Ergebnis vom Typ `bool` bekommen: `<`, `<=`, `==`, `!=`, `>`, `>=`
- `and`, `or` und `not` stellen wir uns (auch wenn das in Python nicht ganz richtig ist) als zwei- bzw. einstellige Operatoren mit Operanden und Ergebnis vom Typ `bool` vor

Vereinfachen Sie folgende Ausdrücke (`a`, `b` und `c` seien dabei Variablen mit einem Wert vom Typ `int`):

- a) `True and False`
- b) `a < b or a == b`
- c) `a < b or a > b`
- d) `not (a <= b and a >= b)`
- e) `a > b == True`
- f) `a < b and c > 0 or a > b and c > 0`